ROOMWISE / 01

Implementation and database report

RoomWise

A simple guide to the Java campus room finder

How the application works, how Java objects connect to MySQL, and how to demonstrate the project confidently.

Report scope	What you will learn
Application flow	Startup, room availability, class searches, and booking.
Core OOP concepts	Encapsulation, inheritance, polymorphism, abstraction, packages, and exception handling.
Database design	Three tables, their keys, constraints, relationships, and an ER diagram.
JDBC and CRUD	Connection setup, parameterized queries, object mapping, and implemented/missing operations.
Demonstration	A practical walkthrough, evidence checklist, and completion plan.

Read this distinction first

This report describes the actual source code reviewed on **9 November 2026**. It does not claim that every assignment requirement is complete. Room searches and booking creation exist. Application-level Update and Delete operations, domain inheritance, and stronger concurrent booking protection still need implementation.

Examples marked **proposed** are recommendations, not features already present. Database diagrams describe the actual checked-in schema unless explicitly labelled otherwise.

The project in simple words

RoomWise answers a practical question: **Which campus room is free at a particular day and time, and how long will it stay free?** It combines the weekly college timetable with bookings made for a specific calendar date.

Two ways to use the same data

Interface	What it currently does
Swing window	Select a weekday/time; click Free or Occupied. Results appear in a table.
Console menu	Find free rooms, list campus rooms, create a booking starting now, or find classes overlapping a time range.

Example: a student needs a room at 10:00

Assume Room A is free at 10:00 and its next class begins at 11:00. RoomWise can show that the room is available until 11:00. If a date-specific booking starts at 10:30, the displayed free period ends at 10:30 instead. These names and times are illustrative, not a live query result.

Weekly timetable versus calendar booking

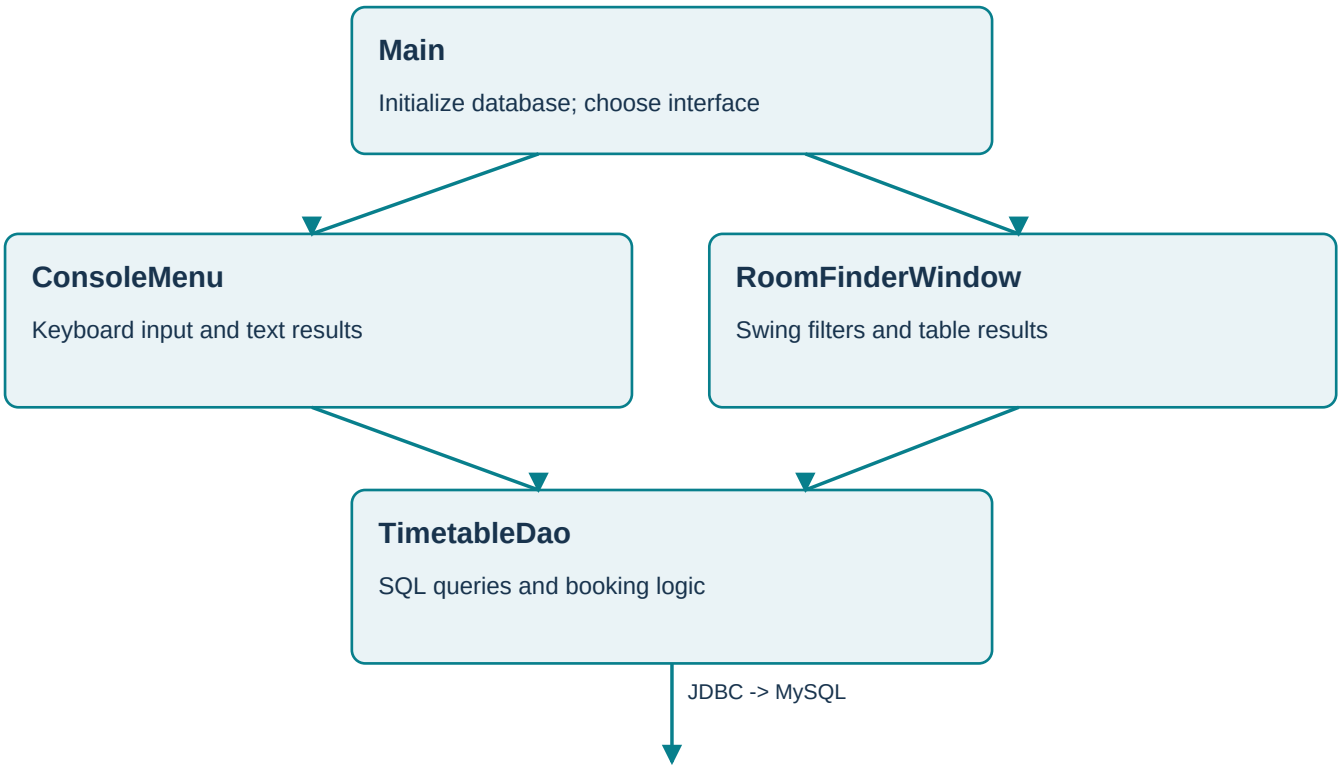
Timetable: a MONDAY entry is treated as repeating every Monday. **Booking:** an entry has a booking_date and applies only to that date. Selecting a weekday in the UI resolves to that weekday in the current week, using the computer clock.

Reading guide

Pages	Topic
3-4	Program structure and OOP evidence
5-7	ER diagram, data dictionary, and constraints
8-10	JDBC, CRUD, availability, and booking transactions
11-12	Demonstration and completion checklist

The source folder contains editable .java files. Maven temporarily generates .class files under target; these are executable bytecode, not a second source codebase. The Windows launcher cleans target after the application exits.

How the parts work together



Actual architecture: both interfaces call the DAO directly. There is no separate business-service layer or DAO interface in this version. DatabaseConnection supplies JDBC connections; DatabaseInitializer prepares the schema and seed data.

Files in src/main/java/roomwise	Responsibility
Main.java	Runs initialization, verifies schema, then opens the chosen interface.
TimetableDao.java	Reads rooms/classes/availability and inserts bookings.
DatabaseConnection.java / DatabaseInitializer.java	Load connection settings and initialize database resources.
Room.java / FreeRoom.java / ClassDetails.java / OccupiedSlot.java	Small records that carry query results as Java objects.
DatabaseException.java	Wraps database or configuration failures with context.

The DAO converts database rows into objects. The interface reads those objects to produce results; it does not need to understand SQL column positions or JDBC connection details.

ROOMWISE / 04

Core OOP concepts: actual evidence

Concept in simple words	Evidence in this code	Status
Encapsulation: keep data and internal details controlled.	Records hold private final components with accessors such as Room.id(). UI fields and helper methods are private. Record constructors do not add domain validation.	Present, basic
Inheritance: reuse behavior from a parent class.	RoomFinderWindow extends JFrame. DatabaseException extends RuntimeException. No Room/Classroom/Lab or User/Faculty domain hierarchy exists.	Framework examples only
Polymorphism: one method call can use a specialized implementation.	An anonymous DefaultTableModel overrides isCellEditable(); Swing uses it to make the table read-only. JDBC variables use interfaces backed by driver implementations.	Present, limited
Abstraction: offer a simple operation while hiding details.	findFreeRooms() hides SQL from the interface; DatabaseConnection.open() hides configuration. No custom abstract domain class or DAO interface exists.	Present as API hiding
Packages: group related Java types.	All 11 types declare package roomwise. There are no separate model, service, dao, or ui packages.	One package
Exception handling: report failures and release resources.	DatabaseException wraps SQL/IO errors. try-with-resources closes JDBC objects. UI catches input/runtime errors; some console database failures can propagate.	Present, uneven coverage

Two small examples you can explain

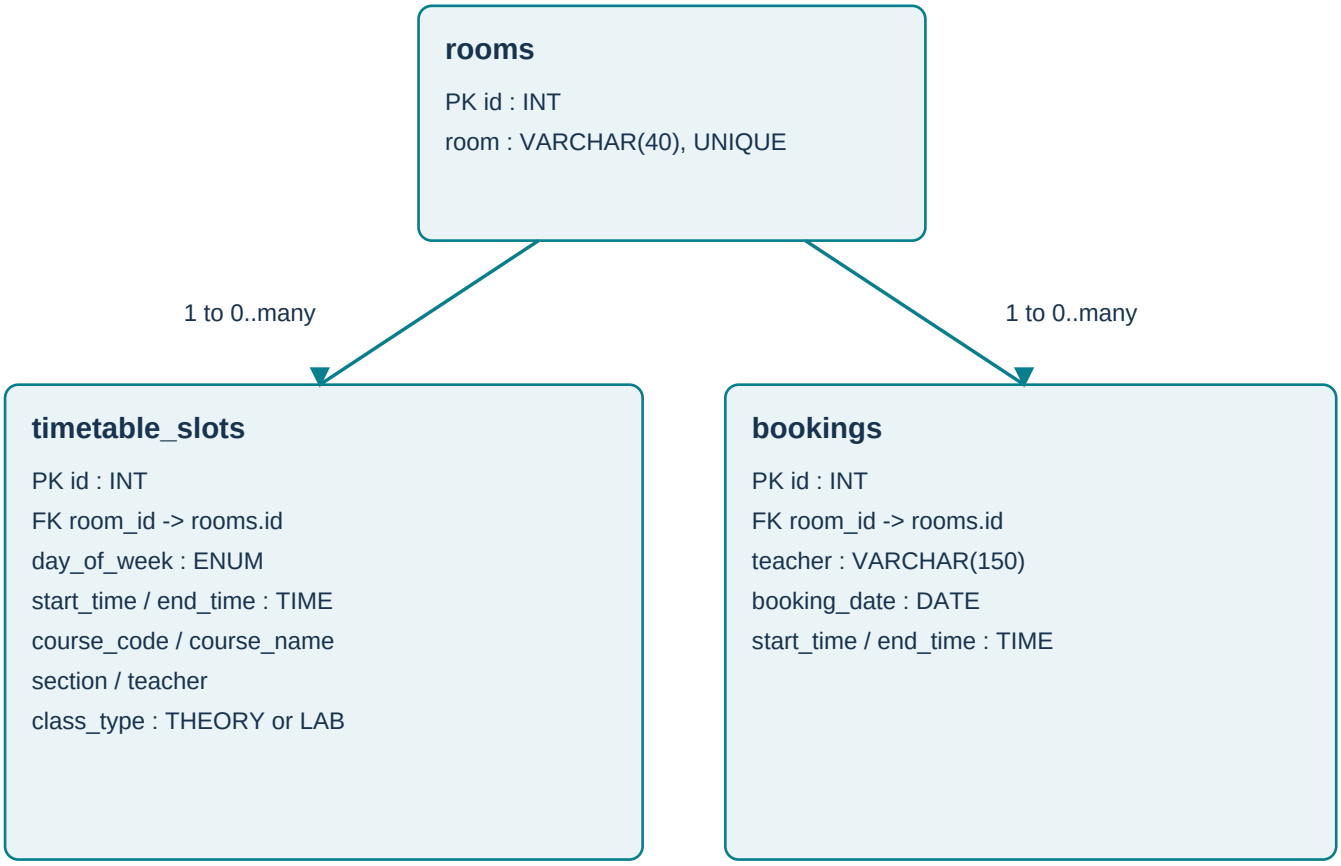
```
record Room(int id, String name) { }
// Data is read through id() and name(); no setters.

@Override
public boolean isCellEditable(int row, int column) {
    return false;
}
```

Do not describe these records as a complete domain inheritance model. They are final data carriers. Method overriding is demonstrated above; custom method overloading is not demonstrated in the present application.

Database diagram: the actual schema

The database is **roomwise_db**. It has three application tables. PK means primary key (a row identifier). FK means foreign key (a reference to a row in another table).



What the relationships mean

One room can have zero, one, or many timetable entries. One room can also have zero, one, or many bookings. Each timetable entry and each booking must reference exactly one existing room, because room_id is a non-null foreign key.

There is **no direct foreign key** between bookings and timetable_slots. Their shared room_id and time ranges are compared by application queries to decide whether a booking conflicts.

What is deliberately absent from this diagram

There are no users, faculty, buildings, courses, or equipment tables in the current schema. Teacher and course details are stored as text in the rows. A richer design may add these tables, but they must not be presented as already implemented.

Data dictionary: rooms and timetable

rooms: the room catalogue

Column	SQL definition	Meaning
id	INT, AUTO_INCREMENT, PK	Unique room identifier; referenced by both child tables.
room	VARCHAR(40), NOT NULL, UNIQUE	Display label. Two room rows cannot use the same label.

timetable_slots: repeating weekly classes

Column	SQL type	Meaning / rule
id	INT, AUTO_INCREMENT, PK	Unique timetable row identifier.
room_id	INT, FK, NOT NULL	Must reference rooms.id.
day_of_week	ENUM, NOT NULL	MONDAY through SUNDAY; weekly recurrence.
start_time	TIME, NOT NULL	Beginning of the scheduled class.
end_time	TIME, NOT NULL	Must be later than start_time.
course_code	VARCHAR(40), NOT NULL	Course code retained from source data.
course_name	VARCHAR(180), NOT NULL	Course title; repeated across timetable entries.
section	VARCHAR(180), NOT NULL	Section/group description.
teacher	VARCHAR(150), NOT NULL	Teacher name as text; not a user-table reference.
class_type	ENUM, NOT NULL	THEORY or LAB.

The source JSON exports are retained under database/data. The initializer expects **4,167 timetable records**. This is a constant in the code, not a fresh database count measured for this report.

Bookings, integrity, and design quality

bookings column	SQL definition	Meaning
id	INT, AUTO_INCREMENT, PK	Identifies a booking.
teacher	VARCHAR(150), NOT NULL	Name supplied in the console.
room_id	INT, NOT NULL, FK	References rooms.id.
booking_date	DATE, NOT NULL	Specific calendar date.
start_time / end_time	TIME, NOT NULL	Requested interval; end must be later than start.

Rules enforced by the schema

Primary keys: identify individual rows. **Foreign keys:** prevent references to rooms that do not exist. **UNIQUE:** avoids duplicate room labels. **NOT NULL:** requires fields to have a value. The named CHECK constraints require end_time > start_time; their enforcement depends on a compatible MySQL server version.

Deleting a room: timetable rows use ON DELETE CASCADE, so they are removed with the room. Bookings use ON DELETE RESTRICT, so an existing booking prevents deleting its room. These database rules do not provide a Delete button in the application.

Indexes used for searches

```
idx_slots_day_room_time
  (day_of_week, room_id, start_time, end_time)

idx_bookings_date_room_time
  (booking_date, room_id, start_time, end_time)
```

Indexes help MySQL locate relevant days/dates and rooms without always examining every row. They are not uniqueness rules and do not prevent overlapping bookings.

Strengths and remaining improvements

The room catalogue is separated from scheduling rows, so a room label is stored once. However, teacher names and course details repeat. This is a simple design, not a fully normalized faculty/course model. A proposed improvement is to introduce faculty and course tables with their own keys, after accounting for source-data collisions.

ROOMWISE / 08

Database connectivity and object mapping

What JDBC means here

JDBC is the Java API used to talk to a relational database. Maven supplies MySQL Connector/J. DatabaseConnection.open() reads db.properties from the classpath, obtains db.url, db.user and db.password, adds connection/socket timeouts, and calls DriverManager.getConnection(). Credentials are intentionally omitted from this report.

The read operation, step by step

Step	Code responsibility
1. Request	ConsoleMenu asks dao.findAllRooms().
2. Connect	DatabaseConnection.open() returns a JDBC Connection.
3. Prepare and execute	The DAO prepares SELECT id, room FROM rooms ORDER BY room and calls executeQuery().
4. Convert rows	Each ResultSet row becomes new Room(id, roomLabel).
5. Display	The interface reads room.id() and room.name() to show results.
6. Close	try-with-resources closes the ResultSet, statement, and connection.

```
while (results.next()) {
    rooms.add(new Room(results.getInt("id"),
                       results.getString("room")));
}
```

Why PreparedStatement is useful

Values such as weekday, date, and time are supplied with setString(), setInt(), and setObject(). The query uses ? placeholders instead of joining user input into SQL. This separates data from SQL syntax and lets the driver handle value conversion.

Errors and startup behavior

SQL/IO failures are wrapped in DatabaseException with their cause. Startup acquires a named MySQL initialization lock, checks the schema and timetable, and may reload SQL. The schema script drops/recreates tables: if the record count or startup checks fail, existing bookings can be lost during reinitialization. Do not edit the seeded timetable casually before a demo.

CRUD: what exists and what is missing

CRUD means Create, Read, Update, and Delete. Reading a row and creating a booking are implemented, but **full CRUD is not yet complete**. Startup seed scripts are not a replacement for user-facing management operations.

Operation	Current evidence	Status
Create	Console option 3 -> TimetableDao.book() -> INSERT INTO bookings. Initializer also loads seed data.	Booking creation exists; no room-management screen.
Read	findAllRooms(), findFreeRooms(), findClasses(), findOccupiedAt(); booking data is read inside availability/conflict queries.	Implemented queries; no dedicated booking-list screen.
Update	No DAO method or UI flow updates a room, timetable entry, or booking.	Not implemented.
Delete	No DAO method or UI flow deletes a room, timetable entry, or booking.	Not implemented.

Proposed examples for completing CRUD

```
-- Proposed SQL; NOT wired into the application
UPDATE bookings SET end_time = ? WHERE id = ?;
DELETE FROM bookings WHERE id = ?;
```

A proper Update flow must load the booking, validate the new interval, check timetable and other-booking conflicts while excluding its own ID, then save in a transaction. It must report whether exactly one row changed. A Delete flow should show the selected booking, request confirmation, delete by ID, and report a missing ID clearly.

Database capability versus application functionality

MySQL supports UPDATE and DELETE, and a developer can type them in Workbench. That alone does not demonstrate integrated application CRUD. To meet the requirement, Java methods and an interface must let a user perform the operations and observe the result.

Availability and booking integration

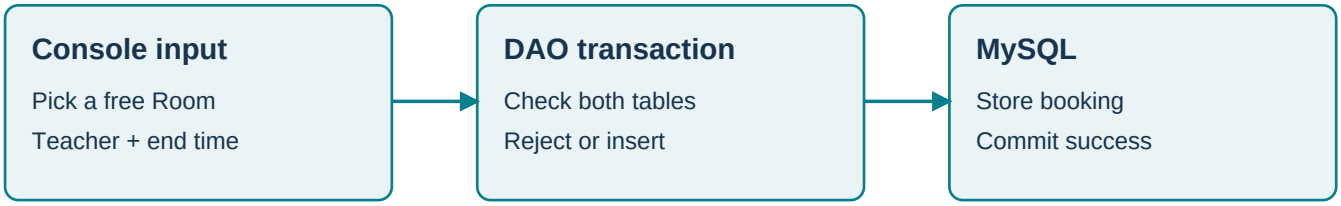
How a room is classified as free

findFreeRooms() rejects rooms with an active weekly class or a booking on the chosen date. Active means start_time <= selectedTime and end_time > selectedTime. For each remaining room, it finds the earliest future class or booking start; that becomes freeUntil. With no later event, the result uses 23:59:59 as end of day.

How overlapping periods are found

```
existingStart < requestedEnd
AND existingEnd > requestedStart
```

For example, an existing 10:00-11:00 class conflicts with 10:30-11:30. A request starting at exactly 11:00 does not conflict with that class. The intervals allow one activity to begin when another ends.



ConsoleMenu validates the selection, nonblank teacher name, and end time. It passes a Room object to book(). The DAO extracts room.id(), binds it to the SQL parameters, checks conflicts, inserts the row, and commits. This is a concrete example of Java objects and database operations working together.

Limits you must explain honestly

The transaction disables auto-commit and rolls back on a detected conflict, but it does not lock a shared room row before checking. Two simultaneous requests can both see no conflict. The initialization lock does not protect bookings. There is also no cross-room check preventing the same teacher from being double-booked.

Proposed fix: use a consistent per-room locking transaction for every booking writer, then recheck and insert; add coordinated teacher checks if required. Also add explicit rollback for SQL failures. The Occupied view currently shows timetable class rows only, although Free considers bookings too; its row count is not necessarily a distinct-room count.

A practical demonstration script

Use the existing project without pretending that missing features are complete. Expected results below are a walkthrough; live counts depend on the database, weekday, and time.

Step	What to do	What to explain
1. Show structure	Open src/main/java/roomwise and database/schema.sql.	One source file per type; SQL kept separately from source.
2. Start console	Run run_roomwise.bat console with MySQL running.	Maven builds; initialization checks data; Main opens ConsoleMenu.
3. Read rooms	Choose 2.	DAO SELECT maps each row to a Room record.
4. Find availability	Choose 1; enter a weekday and time such as MONDAY, 9:00.	Weekly classes plus date-specific bookings determine availability.
5. Show window	Exit with 0; run run_roomwise.bat swing. Choose Free / Occupied.	Both UIs reuse TimetableDao. The table is read-only through an override.
6. Create a booking	In a disposable demo database, choose console option 3. Select a currently free room, teacher, and valid end time.	Console validates; DAO binds a Room ID and writes a booking transaction.
7. Verify booking	Query the latest bookings in Workbench; repeat availability.	A stored booking should exclude its room during its interval.
8. Explain gaps	Show the CRUD status and completion checklist.	Update/Delete and domain-level OOP additions are future work.

```
SELECT id, teacher, room_id, booking_date,
       start_time, end_time
FROM bookings ORDER BY id DESC LIMIT 5;
```

Use a disposable database for booking demonstrations and destructive SQL. Startup may reset altered seed data. No booking was created for this report.

Evidence to collect during your demo

Capture room results, a booking message, its database row, and an input rejection. Demonstrate Update/Delete only after implementing them.

Completion checklist and source guide

What to implement before claiming full compliance

Requirement	Concrete next step
Domain inheritance and abstraction	Proposed: an abstract Room base with Classroom/Lab subclasses, if the domain needs different behavior. Current Room is a record and cannot be extended.
Domain polymorphism	Proposed: override an operation such as allowedPurposes() and call it through a Room reference; demonstrate different outcomes.
Layered packages	Proposed: model, dao, service, exception, util, and ui packages; put business validation in a service shared by both UIs.
Complete CRUD	Add tested create/read/update/delete methods and UI flows for required entities; include row-count and missing-record handling.
Safe booking	Add coordinated locking, explicit rollback paths, cross-room teacher checks, and two-client concurrency tests.
Safer initialization	Replace count-triggered destructive reload with explicit setup/migration and an intentional reset action.
Demonstrable tests	Test overlap boundaries, invalid intervals, FK rejection, CRUD persistence, and simultaneous booking attempts.

Where each explanation was verified

All paths below are relative to RoomWise. These local project files are the primary evidence; no external repository or old report was used to invent missing features.

Source	Evidence
database/schema.sql	Tables, column types, PK/FK constraints, delete rules, indexes.
src/main/java/roomwise/TimetableDao.java	Reads, mapping, overlap queries, booking insert/transaction.
DatabaseConnection.java / DatabaseInitializer.java	JDBC configuration and startup data checks; same Java folder.
ConsoleMenu.java / RoomFinderWindow.java / Main.java	Actual user flows, validation, Swing inheritance/override, startup.
Room.java / FreeRoom.java / ClassDetails.java / OccupiedSlot.java / DatabaseException.java	Data carriers and exception class; same Java folder.
pom.xml / run_roomwise.bat	Java 17 target, MySQL dependency, build/run and output cleanup.

Verification boundary: the prior project check successfully built and started the console against local MySQL. This report is based on source inspection; it does not claim new Update/Delete, booking-write, concurrency, or GUI tests. No application code or database records were changed for this report.